

Transcript

Module 6: Week 7: Introduction to Python - Part 2

Video 1: Introduction to Python Libraries

Python provides many features commonly associated with a high-level programming language such as variables and operators, control flow statements, functions and data structures. It can be readily used for creating software applications and general-purpose software engineering. However, for domain-specific scientific computing especially data analytics and machine learning modelling, the built-in data structures provided by Python are limited in terms of their ability to store and manipulate a two-dimensional dataset.

In this module, we will explore some commonly used Python libraries that can overcome the limitations of Python's built-in data structures. These libraries also provide powerful capability to perform essential tasks in data analytics and machine learning modelling such as data preparation, descriptive analytics and data visualisation. More importantly, these libraries are tightly integrated with each other to afford a seamless development environment to data scientist as well as Artificial Intelligence and machine learning engineers. Let's get started on this exciting learning journey.

Video 2: Overview of Python Libraries for Data Science and AI

In this video, you will be provided with a broad overview of popular Python libraries for data analytics and machine learning modelling. In the subsequent videos of this module, we will deep dive into each of these libraries. Let's begin with NumPy. NumPy is an essential package for scientific computing in Python. It forms the basis for many other mathematical and scientific Python packages. For example, the SciPy library is a collection of mathematical algorithms and convenience functions built on NumPy. The entire NumPy library is based on one main data structure known as the ndarray, which stands for multidimensional array. The ndarray data structure addresses several limitations of Python's built-in basic data structures. The NumPy library also provides methods that operate on the ndarray data structure to support calculation of multidimensional arrays and large arrays.

Next, Pandas is a Python library for performing highly specialised data analysis tasks. It provides a single library for data analysts and AI/ML engineers to easily perform data preparation that entails extracting data, processing data and manipulating data. In particular, it provides support for representing and storing two-dimensional datasets via two new data structures, namely series and dataframe. Collectively, these two data structures provide data analysts and AI/ML engineers with the data manipulation capability equivalent to SQL-based relational database but within the Python environment itself. There are several advantages of working with Pandas. First, Pandas negates the need to move data into and out from the database to perform data preparation. Second, the prepared data in a Pandas DataFrame can be feed into downstream libraries for data visualisation and machine learning modelling.

Matplotlib is a Python library for creating two-dimensional charts. These charts are very useful for data visualisation to help data analysts and AI/ML engineers better understand the nature of relationships among variables in the dataset. Some of the key features of Matplotlib include simplicity of usage, gradual development approach and good control over graphics element. In sum, Matplotlib allows you to easily create an initial chart with default elements and then gradually customise the chart to your exact requirements.

More importantly, Matplotlib is tightly integrated with the other libraries such as Pandas. For example, you can create a chart using data stored in a Pandas dataframe without having to use the Matplotlib library directly. Finally, Scikit-Learn is a Python library that integrates many machine learning algorithms from supervised learning to unsupervised learning. Scikit-Learn allows AI/ML engineers to easily train, evaluate and tune machine learning models using data stored and prepared with a Pandas dataframe. The tight integration among these Python libraries creates a seamless AI workflow that significantly improves the productivity and efficacy of the solutioning process. Beyond Scikit-Learn, TensorFlow is a very popular deep learning framework for the Python ecosystem. With TensorFlow, AI/ML engineers can readily train modern deep learning models such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Generative Adversarial Networks (GANs).

If you want to delve deeper into unstructured data, Python also offers many excellent options. For computer vision tasks, KerasCV and OpenCV are great starting ground. For natural language processing and text analytics, you can consider NLTK and SpaCy. Beyond machine learning modeling, you will be amazed just how much you can do with Python. For big data processing, PySpark is the Python API for Apache Spark. It enables you to perform in-memory, real-time, large-scale data processing in a distributed environment. For managing the machine learning lifecycle, model deployment and MLOps, you can tap onto the MLflow platform. In gist Python is the definitive language for AI with a huge ecosystem of libraries supporting you at every step of the AI solutioning process. I will strongly encourage you to explore on your own starting with the videos in this module.

Video 3: Introduction to NumPy ndarray

This video explains to you the ndarray data structure in the NumPy library. The ndarray is a multidimensional array that is homogenous in nature. This means that all items in the ndarray are of the same type and the same size. In terms of representing and storing a real-world dataset, this homogenous characteristic may not be ideal. This is because most datasets are of a mixed type with some variables being integer numbers (e.g., age and heart rate), floating-point numbers (e.g., price and temperature) and text (e.g., gender and race). The NumPy library also provides many methods for operating on the ndarray object to support calculations of multidimensional arrays and large arrays without the need for you to write your own code. The number of dimensions and items in an array is defined by its shape. More specifically, the size of each dimension is specified by a tuple of N-positive integers. The dimensions are defined as axes and the number of axes is known as the rank. The data type is specified by the dtype attribute. The array() function is used to define a new ndarray by passing a Python list containing the elements to be included.

In this example, the ndarray object is two-dimensional and thus has a ndim of 2, corresponding to the number of axes or rank. The size is 4, which is equivalent to the length of the array. The shape of the ndarray is (2,2). Multiplying all numbers in the tuple will give you the size of the ndarray. Observe that there is only one dtype attribute for the entire ndarray since it is homogeneous. In this example, dtype is float64. NumPy ndarray can contain a wide variety of data types including strings. But again, do take note that it is homogenous. Let's explore how to perform some basic operations on a ndarray object. Arithmetic operations can be performed with arithmetic operators between a ndarray and a scalar value or another ndarray. Common operators include addition or multiplication. Operations between arrays are element-wise, that is, the operators are applied only between corresponding elements that occupy the same position. The end result will be a new array containing the results in the same location of the operands. Arithmetic operations can be applied to ndarray of a higher dimension too.

This example demonstrates both types of arithmetic operation that is, with a scalar value and with another ndarray. Matrix product that applies to two ndarrays can be done with the `dot()` function. In this example, the matrix product involves two-dimensional ndarrays. You can create a two-dimensional ndarray by reshaping from a one-dimensional ndarray. Matrix product can also be performed between ndarrays of higher dimensions. Just imagine how much code you will need to write to perform matrix product without NumPy. To perform indexing and iterating through an ndarray, you can use the square brackets notation. Just like Python list, NumPy ndarray uses zero-based indexing. Indexing allows the users to extract one or more values and to assign new value to an element in the ndarray. Nested for loops can be used to iterate through a higher dimensional ndarray. This example demonstrates indexing of a two-dimensional ndarray. Observe that there are two different approaches of writing the square bracket notation. Also observe that it is easy to traverse a ndarray row-wise and cell-wise.

But how about traversing a ndarray column-wise? Recall that with Python built-in basic data structure, you cannot easily perform a column-wise traversal. But with an NumPy ndarray, you can perform a transposition to swap the axes of the ndarray. For a two-dimensional array, this will swap the rows and the columns. In this example, after the transposition, the same row traversal (in green colour) will thus be equivalent to a column traversal in blue color. The red colour values represent the first column in the original two-dimensional ndarray. Other than indexing, it is also possible to perform slicing of a ndarray using the colon indexing syntax. A higher dimensional ndarray allows you to slice one or more axes. In this example, slicing is performed on a one-dimensional ndarray and a two-dimensional ndarray. Observe the flexibility provided by ndarray for indexing and slicing the required elements.

Video 4: Data Processing with NumPy ndarray

In this video, we will progress to perform more advanced data processing operations on a ndarray data structure. Other than using zero-based index numbers for indexing, we can also use a Boolean array. The Boolean array must be of the same length as the axis it is indexing. In this example, we generate a Boolean array from the names ndarray of length 7 to identify those items matching 'Bob'. This Boolean array, also of length 7, is then used to slice the numerical data in the corresponding position within the data ndarray (in blue). Boolean indexing can make use of relational operators as required. In fact, we can include multiple Boolean conditions using the logical operators.

In this example, we slice the numerical data corresponding to 'Bob' and 'Will' (in blue). Boolean indexing can also be used to set or assign values to a ndarray conditionally. In this example, we reset negative numerical values to zero using a Boolean expression written with a relational operator. As you will have observed, the NumPy ndarray provides very flexible and powerful indexing and slicing support to manipulate data. Next, let's examine NumPy universal function of `ufunc`. A universal function is one that performs elementwise operations on data stored in a ndarray data structure. Universal functions are essentially fast vectorised wrapper for simple functions that take one or more scalar values and produce one or more scalar results. Most universal functions are simple elementwise transformation such as `sqrt` or exponentiation.

NumPy universal functions are written in C and thus offer faster performance compared to you writing your own Python code. Other universal functions such as `add` or `maximum` take two ndarrays and return a single ndarray as the results. A universal function such as `modf` can also return multiple ndarrays, in this case, one for functions and another for integral parts of a floating-point array. This example demonstrates the use of the `sqrt` and exponentiation universal functions. NumPy also allows us to express many types of data processing tasks as concise array expressions.

This negates the need to write iterative control flow statements in Python. This practice of replacing loops with array expression is known as vectorisation. Vectorisation is generally more efficient as vectorised array operations is often much faster than their pure Python equivalents. This is especially so for numerical computation. In this example, we can evaluate the function $\sqrt{x^2+y^2}$ across a regular grid of values easily using vectorisation. NumPy also allows you to express conditional logic as ndarray operations. The `np.where` function is a vectorised version of the Python ternary conditional expression `x if condition else y`. Suppose we have a Boolean ndarray and two ndarrays of values. If we want to take a value from the first ndarray x-array whenever the corresponding value in condition is true and otherwise from the second ndarray y-array.

In Python, we can use a list comprehension or iterative control flow statement to perform this task although there are two problems. First, it will not be very fast for large ndarray. Second, it will not work with multidimensional ndarray. With `np.where`, we can write this concisely as `result = np.where(cond, xarr, yarr)`. NumPy provides a set of mathematical functions for computing statistics about an entire ndarray of data along an axis. These are mainly aggregations such as `sum`, `mean` and `std`. This table list down the basic array statistical methods that are available. In this example, we first compute the mean and sum of all values in the two-dimensional ndarray. This is followed by the computation of the mean on axis 1 of the ndarray, that is, within each row of values.

NumPy also provides support for linear algebra on the ndarray object. These include matrix multiplication, decomposition, determinants and other square matrix math operations. This table lists down some of the commonly used functions in the numpy linear algebra package. Finally, the `numpy.random` module supplements the built-in Python random functions to efficiently generate an entire ndarray of sample values from many kinds of probability distributions. For example, you can also obtain a 4 by 4 ndarray of samples from the standard normal distribution using the `normal` function.

In contrast, Python's built-in random module only samples one value at a time. Functions in the `numpy.random` package is certainly much faster for generating very large samples. This table list down the commonly used `numpy.random` functions. This example demonstrates the generation of a 4 by 4 ndarray of samples from the standard normal distribution. Observe that the mean is about zero and the standard deviation is about one. With this, we have reached the end of this video. NumPy provides many fast vectorisation functions to operate on data stored in a ndarray data structure. This contrasts with the use of the Python built-in basic data structures in which you will need to write all the required code by yourself. With the use of NumPy, scientific computing is certainly much easier and faster for data analysts and AI/ML engineers.

Video 5: Introduction to Pandas Series

In the previous videos of this module, we have discussed at length about the NumPy ndarray data structure. The ndarray data structure offers significant improvement in terms of capability and performance relative to the Python built-in basic data structures. However, the ndarray data structures also has its own limitations. First ndarray is a homogeneous data structure and all items must be of the same data type. This is not reflective of real-world datasets, which are typically mixed type. Second, items stored in an ndarray can only be referenced using zero-based index numbers. As the number of columns, or variables, and rows in a dataset increases, it becomes increasingly challenging to reference data using zero-based index numbers. Ideally, we want to be able to use text label to reference the variables and rows instead.

For example, instead of referencing our dataset as variable 30 or record 1,000, we will prefer to use variable "age" and record associated with a particular alphanumeric student number. The Pandas Library provides two new data structures to address these limitations of ndarray, namely Series and DataFrame. In this video, we will begin by examining the Pandas Series data structure first. Series is a one-dimensional data structure. In the next video, we will then look at how multiple Series data structures can be combined to create a two-dimensional DataFrame. The Pandas DataFrame is the primary data structure that is used for data preparation as well as the subsequent data analysis and machine learning modelling. Pandas Series is essentially similar to a Python list but with some additional features.

More specifically, the internal structure of a Series data structure consists of two lists that are associated with each other. The main list holds the data, which can be any NumPy type. Each piece of data or element is associated with a label. The label is contained within the other index list. As shown in this diagram, by default, the label is a zero-based index number. But in this case, the zero-based index number can be overridden with a more meaningful text label. A Series can be created by calling the Series() constructor and passing in a Python list containing the values to be included in the Series. To override the default zero-based index number with a more meaningful text label, the index option can be used to pass in a list of strings. Observe the difference between the label list of the two Series data structures. Note that all values in a Series must be homogenous, that is, of the same data type.

Internal elements in a Series can be selected using the square brackets notation with either a zero-based index number or a text label, if one is defined with the index option. When using a zero-based index number, you should use the iloc property. Series thus provides a good degree of flexibility in indexing elements. Observe that multiple values can be selected and that values can also be assigned to elements with both a zero-based index number and text label. Finally, individual elements can be filtered by their value using the NumPy filter syntax. We can perform various operations and mathematical functions that are applicable to NumPy ndarray to Pandas Series too. Beyond these basic operations, we can also evaluate its values within a series to provide information on the unique values, count of duplicate values and whether a value is present or not. This information is useful for descriptive analytics.

For example, if a string variable contains too many unique values, it should not be considered as a structured categorical variable but rather unstructured text. Observe that Series and dict are actually very similar in the general structure. Thus, Series can be created from a dict object. Finally, since we can perform arithmetic operation between a Series and a scalar value, we can also do that between two Series. In this example, we perform the arithmetic addition operation between two Series. Observe that some of the elements in the resulting Series contain the special value of NaN, which denotes 'Not a Number'. NaN represents missing value and occurs when a specific label is missing from either one of the two Series being added. For example, 'black' is in the second series but not the first one. Knowledge of missing values in a dataset is important and Pandas can provide this information automatically. This concludes our discussion on Series.

Video 6: Introduction to Pandas DataFrame

A Pandas DataFrame is essentially a two-dimensional data structure resembling a database table and spreadsheet. DataFrame extends the series to multiple dimensions or columns. It is essentially a collection of columns, each of which can contain values of different data types. Thus, DataFrame can represent and store a real-world mixed-type dataset. Recall that a series has an index list containing labels associated with each element.

A DataFrame has two index lists. The first index list is associated with the rows and serves a similar function as the index list in a Series. But in this case, each label is associated with multiple values in that row. Think of this as the row header in a spreadsheet or primary key in a relational database table. The second index list contains a collection of labels, each associated with a particular column or variable. Think of this as the column header in a spreadsheet or column or attribute name in a relational database table.

In this example, the DataFrame contains three columns, namely color, object and price. Observe that price is a floating-point number whereas color and object are string. Each label or zero-based index number on the row header is associated with all three values of colour, object and price for a particular row. Another way to look at a DataFrame is to think of it as a dict of Series. The keys of the dict are the column names. The values are the Series. Furthermore, all elements of each series are mapped according to a list of labels called the index. The most common way to define a DataFrame is to pass a dict object to the DataFrame() constructor. The dict object contains a key for each column to be defined together with a list of values. This approach is quite similar to how we define a Series but now we are essentially defining multiple Series objects to make up the DataFrame.

Let's move on to find out how to select elements in a DataFrame. To select the contents of a column, we can use the column name as an index or use the column name as an attribute of the DataFrame. To select the content of a row, we can use the iloc attribute with a positional index or the loc attribute with the required label index. To select the content of a cell, we combine the iloc or loc attribute with the required column name. We can assign new value to each cell with the assignment operator and the loc attribute of the DataFrame. Arithmetic operations can be performed between two DataFrame objects using the built-in arithmetic methods such as add(), sub(), div() and mul(). In this example, observe that the resulting DataFrame contains the 'NaN' special values. This means that there are missing values in the dataset. For two elements to be added, their column index and row index must exist across both DataFrame objects. Statistics function can also be applied on a DataFrame.

In this example, the DataFrame describe() function can generate the five-number summary descriptive statistics for each of the numerical variables. Observe that Pandas DataFrame addresses all the limitations associated with Python built-in basic data structure and the NumPy ndarray object. With DataFrame, we can store and manipulate a two-dimensional, labelled, mixed type dataset effectively with minimal effort. In the next video, we will find out how to perform more advanced data preparation operations with a DataFrame.

Video 7: Data Preparation with Pandas DataFrame

In this video, you will learn how to perform common data preparation tasks with a Pandas DataFrame. Let's begin with importing data into a DataFrame. We can read data files in various formats into a DataFrame object. For example, the content of a CSV or comma-separated values file can be read in with the read_csv() function and converted to a DataFrame object. The index_col attribute of read_csv() function can be used to designate the row header or primary key. Once data is imported into a DataFrame, we can easily manipulate them with the built-in functions provided by Pandas. We can also import data in other formats such as Excel, SQL and JSON. The read_sql_table() and read_sql_query() functions allow us to directly import data from a relational database. In this example, we import a CSV file into a DataFrame and then use the describe() function to generate descriptive statistics about the dataset.

Observe the difference with and without the use of the `index_col` attribute. In the second DataFrame, the animal column is designated as the row index (in blue colour). Missing data is a common problem in data analytics. Pandas is designed to make missing data handling as easy as possible. Descriptive statistics on DataFrame exclude missing data. Also, recall that missing data are indicated with the special value `NaN`, which denotes 'Not a Number'. Pandas also provides various built-in functions to handle missing data. These functions are listed in the table.

In data analytics, dropping an observation with a missing value in at least one of the variables is known as listwise deletion. The `dropna()` function allows us to specify more specific missing value deletion criteria. This example demonstrates the use of the `dropna()` function. Observe that by default, `dropna()` will lead to an empty DataFrame since each row contains at least one missing value across the five variables. We can define a more stringent criterion for listwise deletion as desired. Also observe that we can replace missing values with a prespecified value such as 0. Finally, the `isnull()` function can be used to identify missing values.

When using the `dropna()` function, extra caution must be taken if using zero-based indexing. This is because the zero-based index number is not automatically reset. You need to use the `reset_index()` function. Observe that it is useful to drop the old row index to avoid confusion. Let's move on to find out how to perform feature engineering by manipulating columns in a DataFrame. A new column can be added by setting the new column name to be equal to an initial/default value, for example, 0. We can also initialise the new column to the null value using the NumPy special value `np.nan`. We can drop a column using the `drop()` function. When dropping column, we need to specify the column key and set the axis attribute to 1.

Setting axis to 0 is used to drop a row, which is the default behavior. The values of a new column can be easily computed from values of other existing columns using the vectorisation approach, without you having to write any iterative control flow statement. In this example, observe the use of the vectorisation approach to create the total and excess columns in the DataFrame. Last but not least, we can write or export data in the DataFrame to a file in various formats. For example, CSV file can be written with the `to_csv()` function. The delimiter can be specified with the separator attribute. This wrap up our discussion on the Pandas DataFrame. Observe how easy it is to prepare data in a DataFrame without the need to involve a relational database. More importantly, the prepared dataset can be directly feed into other Python libraries for data visualisation or machine learning modelling. In the next two videos, we'll explore the use of the Matplotlib library to visualise data in a DataFrame.

Video 8: Introduction to Matplotlib

In this video, you will learn how to use the matplotlib library to create two-dimensional charts for data visualisation using data stored in a Pandas DataFrame. Let's begin with an overview of the matplotlib architecture. Matplotlib provides a set of functions and tools that allow the representation and manipulation of a figure, which is the main object, and its associated internal objects. Other than graphics, Matplotlib also handles events and graphics animation. Thus, Matplotlib can produce interactive charts that can respond to events triggered by keyboard or mouse movement. The architecture of Matplotlib is structured into three layers with unidirectional communication. Each layer can only communicate with the underlying layer. At the lowest level, the backend layer consists of the Matplotlib application programming interface and a set of classes that are used to represent the graphic elements.

The artist layer is an intermediate layer representing all the elements that make up a chart, For example, title, axis, labels, markers, etc.

The scripting layer consists of the Pyplot interface for actual data calculation, analysis and visualisation. Putting all three layers together, the scripting layer is used by the data analyst to create the required objects constituting the chart, which are represented in the artist layer that forms the basis for the backend layer to render the actual chart. At the artist layer, the three main objects in its hierarchy include the figure, Axes, and Axis. Each figure can consist of one or more Axes, with each Axes forming one plot or subplot. Each instance of a chart corresponds to an instance of artist structured in a hierarchy. As shown in this diagram, an axes consists of an x-axis and y-axis, together with the actual plotting element, in this case, a two-dimensional line plot. Each Axis is further broken down into its label and ticks. This hierarchical structure allows every internal object to be scripted and customised to the exact requirements of the data analyst. The Pyplot module is a collection of command- style functions that allow the data analyst to operate or make changes to the figure. For example, to create a new figure.

A simple chart can be created using the Pyplot object's `plot()` function. The `plot()` function uses a default configuration that does not have title, axis label, legend, etc. Observe that the list of integer values passed into the `plot()` function appears on the y-axis. The x-axis corresponds to the zero-based index number of the elements in the list. The default configuration can be changed to obtain the desired chart. In this example, the line plot has been replaced by a red colour dot plot with a proper chart title. The scaling of the x-axis and y-axis can also be customised. Observe that there are two lists of integer values being passed into the `plot()` function. The first list appears on the x-axis and the second list appears on the y-axis.

Matplotlib is tightly integrated with Pandas. To create a simple line plot from data in a Pandas DataFrame, we can use the `DataFrame.plot()` function, which relies on matplotlib. By default, you will get a line plot. To customise the line plot created by the `DataFrame.plot()` function, you will need to use the matplotlib directly. Observe the creation of an annotation on the chart itself. To create a bar plot, you need to set the kind. By default, the index of the DataFrame is a zero-based number and thus the x-axis are the numbers 0 to 4. You can change the index of the DataFrame to the Name column and replot the bar plot. Observe that the chart with name as the x-axis is certainly more informative. By now, you will have noticed that creating two-dimensional charts in Python using matplotlib is very easy given the integration with Pandas.

Video 9: Data Visualisation and Matplotlib

In data analytics and machine learning modelling, exploratory data analysis or EDA is an important step that helps you to understand more about the nature of relationships among variables in the dataset. Through the EDA process, you will be able to identify the most suitable independent variables for predicting the phenomenon of interest, that is the dependent variable, and to find the most appropriate model to fit the data. This video introduces you to the use of the Matplotlib library to create more advanced data visualisation for EDA. Let's begin with the boxplot. The boxplot is a visual summary of the descriptive statistical properties of a variable. In particular, it can be used to depict the quartiles of numerical data. In a boxplot, the box extends from the first quartile to the third quartile values of the data, with a line at the median or the second quartile. The whiskers extend from the edges of the box to show the full range of the data. The position of the whiskers can represent different information. If there is no outlier, the whiskers show the minimum and maximum values of the data.

If there are outliers, the whiskers show the lowest and highest datum that they are still within 1.5 times the interquartile range of the lower and upper quartile, respectively. Any observations that are past the end of the whiskers are considered as outliers.

In this example, we attempt to create two boxplots of the academic grade of students. In the second boxplot, observe that a new grade record of 130 for Charles is deliberately added to simulate an outlier. This outlier can be clearly seen in the corresponding boxplot. We can also create a box plot with categorisation, that is, the data is split using a categorical variable, In this case by Gender. Such a box plot provides additional insights about the data. In this case, observe that female students tend to perform better than male students, but the grades of female students are less consistent. In comparison, the performance of male students is lower than female students, but male students are more consistent. A histogram can be created using the `DataFrame.hist()` function. By default, the histograms for the columns with numeric values will be generated. You can also generate histogram for a numerical column grouped by another column, typically a categorical variable, using the `by` attribute.

In this example, we generated the histogram for number of hours spent by students studying. The first histogram for all students shows that the values of the variable are somewhat normally distributed. The second histogram grouped by gender of students suggests that more male students spend a higher number of hours studying as compared to female students. A pie chart can also be created using the `pyplot.pie()` function. Customisation can be made to the pie chart to make it more meaningful and visually appealing. In this example, the pie chart shows the proportion of demerit points that each student has accumulated. Mel has the highest proportion of demerit points among the five students.

A scatter plot can be created using the `pyplot.scatter()` function. A scatter plot is very useful for visualising the relationship between two numerical variables. This is also known as a bivariate scatter plot. In this example, we created two bivariate scatter plots for the Iris flower dataset. The first scatter plot depicts sepal length by sepal width. The second scatter plot depicts petal length by petal width. The dots are coloured according to the Iris flower species. Observed that petal length and petal width are better independent variables for classifying the Iris flowers as compared to sepal length and sepal width. There is a clear segregation of small, moderate and large petal dimension across the three species of the Iris flowers. This allows the AI/ML engineer to focus on petal length and petal width when training a machine learning model to classify the Iris flowers.

In descriptive analytics, a pivot table can be used to analyse the relationship between two or more numerical variables. In the most basic case, we can analyse the frequency of observations distributed among the variables. This example is based on an automobile dataset. From the pivot table, we can observe the number or frequency of cars for each make and body style. For example, Toyota has the greatest number of vehicles with hatchback being the most common body style in blue colour. Pandas DataFrame also supports the group by operation. A group by operation involves some combination of splitting the object, applying a function, and combining the results. The combined results provide an aggregated comparison of a phenomenon of interest. This example is based on the Titanic shipwreck dataset, which is a classic dataset used for predicting which passengers survived the Titanic shipwreck, using machine learning models.

Observe that by grouping the survival data by sex or gender, we can easily infer that female passengers have a higher likelihood of surviving the shipwreck.

Regrouping the survival data by passenger class suggests that passengers travelling at a higher class, in this case first class, has a higher chance of survival. Suppose you want to train a machine learning model to predict passenger survival, sex and passenger class will likely be included in your model as independent variables.

I hope this video has given you an excellent introduction to using Matplotlib for creating useful data visualisation as part of your exploratory data analysis. Creating data visualisation in Python with the help of libraries such as Matplotlib and Pandas is certainly an easy and useful process.

Video 10: Module Summary

Congratulations on completing this module, You certainly deserve a pat on your back! In this module, we have explored many Python libraries that can overcome the limitations of Python's built-in data structures. These libraries also provide powerful capability to perform essential tasks in data analytics and machine learning modelling such as data preparation, descriptive analytics and data visualisation.

In particular, the libraries are very tightly integrated to provide a seamless and user-friendly environment for you to perform your analysis and modelling tasks. Please continue to explore these libraries further and learn more about working with them. You will find them to be valuable toolkits as you continue your journey towards AI solutioning mastery.

THE END